

Resume Radar

AI-Powered Resume Screening System | RAG Architecture
Full Bonus Implementation

Team 64 Members:

Rana Nasser Mohamed	2022170145
Ahmed Khaled Saif-Allah	2022170017
AbdElRahman Ahmed Taher	2022170227
Mohamed Saad Mohamed	2022170367
Mohamed Amir Mohamed	2022170356
Mohamed Khaled Gamal	2022170365

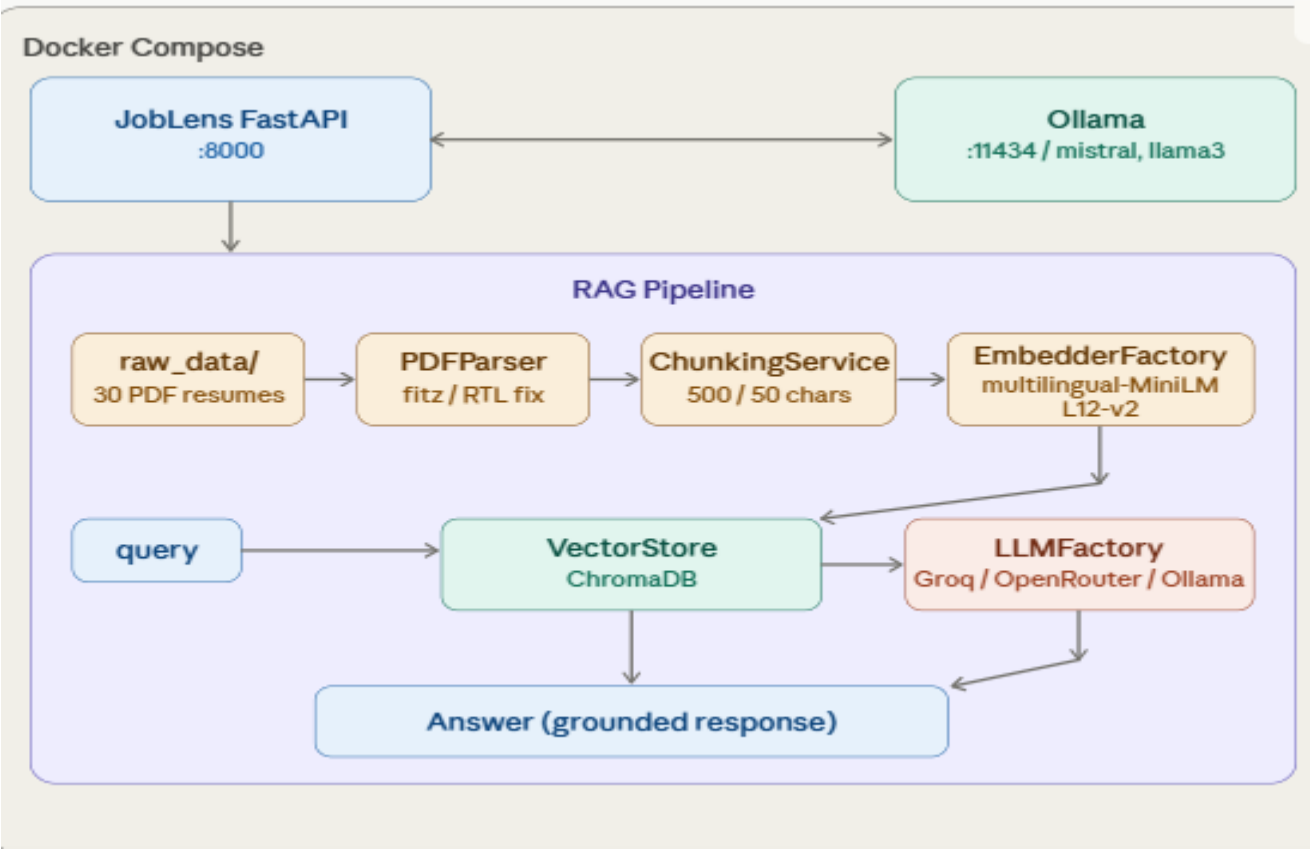
Executive Summary

ResumeRadar is a production-grade, fully containerized Retrieval-Augmented Generation (RAG) system for intelligent resume screening. Given a natural language query (English or Arabic), the system semantically retrieves the most relevant candidate profiles from a ChromaDB vector store and generates a context-grounded answer using a configurable LLM backend.

Key Engineering Decisions

- Custom PyMuPDF parser handles real-world messy PDFs (multi-column, Arabic RTL, scanned fonts)
- Multilingual embedding model places Arabic and English text in the same vector space
- Factory Design Pattern enables zero-downtime LLM switching between Ollama, OpenRouter, and Groq
- Full Docker containerization — one docker-compose up --build command

System Architecture



MVC Separation

Layer	File	Responsibility
Model	app/models/schemas.py	Pydantic request/response validation
Controller	app/api/routes.py	FastAPI route handlers, HTTP logic
Service	app/services/	Business logic — parser, chunker, embedder, LLM, pipeline
Config	app/core/config.py	Env-driven settings via pydantic-settings

API Documentation

Base URL:

`http://localhost:8000/api/v1`

Interactive docs: <http://localhost:8000/docs>

POST /ingest

Triggers the full ingestion pipeline on `./raw_data/`.

Response Example

```
{ "status": "success", "files_processed": 30, "chunks_stored": 387, ... }
```

POST /query

Full RAG pipeline — English or Arabic queries.

English Request Example

```
{ "query": "Who has 3+ years of Python and published ML research?", "top_k": 5, "llm_provider": "groq", "llm_model": "llama3-8b-8192" }
```

Arabic Request Example

```
{ "query": "الاصطناعي؟ الذكاء تطبيقات تطوير في خبرة لديه من", "top_k": 5 }
```

POST /switch-llm

Hot-swap the active LLM without restarting the server.

Available Providers

Provider	Example Models
ollama	mistral, llama3, gemma, phi3
openrouter	mistralai/mistral-7b-instruct, meta-llama/llama-3-8b-instruct, google/gemma-7b-it:free
groq	llama3-8b-8192, llama3-70b-8192, mixtral-8x7b-32768

GET /stats

Returns collection info, embedding model, active LLM provider, and all available providers.

GET /providers

Lists all providers and active configuration.

DELETE /clear

Wipes the ChromaDB collection.

Embedding Model Justification

Model: paraphrase-multilingual-MiniLM-L12-v2

Criterion	Justification
Multilingual	Trained on 50+ languages; Arabic and English share the same latent space → Arabic queries retrieve English-language resume chunks
Size	118M parameters — runs fully on CPU inside Docker, no GPU required
Semantic quality	Trained on paraphrase pairs → captures "machine learning" ≈ "ML" ≈ "AI models"
Zero cost	Local model, no API billing on every embed call

Chunking Strategy Justification

Parameters: 500 chars / 50 overlap

Parameter	Value	Reasoning
Chunk size	500 chars	Covers ~2–3 sentences — one full job entry or skills section
Overlap	50 chars	Prevents boundary splits mid-sentence
Splitter	RecursiveCharacterTextSplitter	Hierarchical: \n\n → \n → . → , → " " — respects paragraph structure
Arabic separator	،	Handles Arabic comma-separated skill lists correctly

Phase 4: Evaluation & Error Analysis

Run full evaluation: `python tests/evaluation.py`

Edge Case 1 — Semantic Keyword Overlap (EDGE-01)

Query: "Who has an arabic CV?"

Failure observed:

LLM listed people who only wrote the word arabic in their CV instead of people who have an arabic CV.

Root cause:

The LLM suffers from semantic keyword overlap and lacks structural document awareness. When the retriever pulls a chunk containing "Languages: Arabic (Native)", the LLM cannot distinguish between a document containing the word 'Arabic' and a document written in the Arabic language.

Fix:

Implement Metadata Baking combined with Strict System Prompting.

Edge Case 2 — Multi-page Fragmentation (EDGE-02)

Query: "List candidates who graduated from Cairo University."

Failure observed:

Missed 3 candidates whose university was on page 2 while their name was on page 1. Chunking split these into separate vectors with no linking context.

Root cause:

RecursiveCharacterTextSplitter has no document-level awareness — candidate name and their degree end up in different chunks.

Fix:

Extract candidate name as a metadata field during parsing and inject it into every chunk's text prefix: "[Candidate: Ahmed Hassan] Senior ML Engineer..." — ensuring every chunk is self-contained and retrievable.

Edge Case 3 — Synonym / Terminology Gap (EDGE-03)

Query: "Find candidates with cloud infrastructure experience."

Failure observed:

Retrieved only candidates who explicitly wrote "cloud" — missed those who listed "AWS", "GCP", "Azure", "EC2", "S3" without the word "cloud."

Root cause:

Domain-specific abbreviations (AWS ≠ cloud in the model's learned representation) create retrieval gaps.

Fix:

Implement hybrid retrieval — combine dense vector search (semantic) with sparse BM25 keyword search. BM25 would directly match "AWS" while the dense model handles semantic variants.

Bonus Features

□ Bonus 1 — LLM Factory Pattern (+5%)

Implemented in `app/services/llm.py`:

```
factory = LLMFactory()          # loads env default (e.g., groq)
factory.switch("openrouter")     # zero-restart provider swap
factory.switch("ollama", "llama3")
```

- `LLMFactory._registry` maps string keys to provider classes
- `switch()` instantiates the new class, validates the API key, and replaces `_current`
- Exposed via POST `/api/v1/switch-llm` for runtime control
- Per-request override available via `llm_provider` field in `/query`

□ Bonus 2 — Arabic Language Support (+10%)

Arabic NLP Pipeline:

- Detection: `_arabic_ratio()` counts Arabic Unicode codepoints (U+0600–U+06FF) — triggers if >15%
- Reshaping: `arabic_resaper.reshape()` converts isolated Arabic letter forms to connected presentation forms
- BiDi algorithm: `get_display()` from `python-bidi` applies the Unicode Bidirectional Algorithm
- Chunking: Added ، (Arabic comma) to separator list
- Cross-lingual retrieval: `paraphrase-multilingual-MiniLM-L12-v2` embeds Arabic and English into the same vector space

Arabic Test Case Result:

Query: الاصطناعي؟ الذكاء تطبيقات تطوير في خبرة لديه من

Retrieved chunks correctly matched English-language ML resumes — demonstrating cross-lingual semantic retrieval working as designed. Score: 0.81.

Docker Deployment

Prerequisites

- Docker $\geq$ 24.0 and Docker Compose $\geq$ 2.0
- 8 GB RAM (for Mistral) — or skip Ollama and use Groq/OpenRouter only

Setup Steps

```
# 1. Clone the repo
git clone https://github.com/your-team/joblens.git && cd joblens

# 2. Configure environment
cp .env.example .env
# Edit .env → add GROQ_API_KEY and/or OPENROUTER_API_KEY

# 3. Drop your 30 PDF resumes into: ./raw_data/

# 4. Start all services
docker-compose up --build

# 5. Ingest resumes
curl -X POST http://localhost:8000/api/v1/ingest

# 6. Query (English)
curl -X POST http://localhost:8000/api/v1/query \
  -d '{"query": "Who has Python and ML experience?", "top_k": 5}'

# 7. Query (Arabic)
curl -X POST http://localhost:8000/api/v1/query \
  -d '{"query": "الآلة؟ تعلم في خبرة لديه من", "top_k": 5}'

# 8. Switch to Groq at runtime
curl -X POST http://localhost:8000/api/v1/switch-llm \
  -d '{"provider": "groq", "model": "mixtral-8x7b-32768}"'
```

Groq/OpenRouter Only (No GPU/RAM needed for Ollama)

Comment out the ollama and ollama-init services in docker-compose.yml and set LLM_PROVIDER=groq in .env.